

Research on Improving Fairness of Linux Scheduler

Wenbo Wu and Xinyu Yao

College of Information System and Management
National University of Defense Technology
Changsha, Hunan Province, China
just4thin@gmail.com 13873120926@139.com

Wei Feng

The 63892th Unit of PLA
Luoyang, Henan Province, China
fw860102@163.com

Yong Chen

Beijing Oil Research Institute
Beijing, China
blankman1983@126.com

Abstract—The Operating System scheduler is designed to allocate the CPU resources appropriately to all processes. The Linux Completely Fair Scheduler (CFS) design ensures fairness among tasks using the thread fair scheduling algorithm. This algorithm ensures allocation of resources based on the number of threads in the system and not within executing programs. Fairness issue of the CFS used in Linux kernel comes up due to the fact that programs with higher number of threads are favored by the scheduler, which based on the number of threads in the system. A novel algorithm as well as its implementation through optimized procedure is proposed as a solution to achieving better fairness by punishing greedy-threaded programs. Several tests are conducted to illustrate fairness issue and to examine the effect of the proposed algorithm.

Index Terms—Linux, Completely Fair Scheduler

I. INTRODUCTION

An operating system is a software application that acts as an interface between user and the computer hardware. Its major responsibilities are to manage and ensure proper operations of the hardware resources[1]. Typical resources are main memories and physical devices. But CPU could be regarded as a resource too, for the scheduler can allocate a task running on it. The main purpose of scheduler is allocating CPU time slice efficiently, and meanwhile offering nice user experience. Namely it considers the balance of interactivity (the capability of user response) and fairness.

Linux kernel keeps developing rapidly in recent years, it introduces a lot of new technologies, and makes a giant improvement in reliability, flexibility and performance[2,3] Especially, Linux achieves continuous innovations and developments in the field of scheduler, which is the key component of any operation system, Early Linux schedulers used minimal designs, the 1.2 Linux scheduler used a circular queue for runnable task management that operated with a round-robin scheduling policy, Linux version 2.2 introduced the idea of scheduling classes, the 2.4 kernel used O(N) scheduler[4]. The early 2.6 scheduler, called the O(1) scheduler, it was much more efficient and much more scalable but it became unwieldy in the kernel, and was fundamentally difficult to manage. Ingo Molnar, the creator of the O(1) scheduler, then developed the CFS (Completely Fair Scheduler)[5,6], and introduced it in the kernel 2.6.23.

Con Kolivas introduces the concept of completely fair into scheduler, named SD (Staircase Scheduler), and its ad-

vanced version called RSDL (The Rotating Staircase Deadline Scheduler). CFS Ingo Molnar then developed the CFS based around some of the ideas from Kolivas' work. Fairness is the most important characteristic of CFS, it did a lot of works to keep scheduling fair, including group scheduling, which is designed to solve the problem of fairness issue caused by multi child processes. However, there is the same problem in multi-threaded applications: one process can create many threads (more than reasonable) to dominate the CPU bandwidth when there are other programs executing concurrently. This fairness/starvation issue is particularly exposed in purely CPU-bound multi-threaded applications.

In order to solve the fairness issue, and keep scheduler algorithm elegant and efficient, we need to modify the CFS.

Contributions:

The main contributions of this paper are: (i) Analysis of Linux CFS scheduler, pointing out its fairness issue. (ii) A solution called dynamic weight completely fair scheduler (DW-CFS) is presented, (iii) obtained the function that suits for the demand of dynamic weight, and (iv) demonstration of its effectiveness through an experimental evaluation.

Organization:

The remaining part of this paper is presented as follows: In Section 2 the analysis and algorithm of CFS is presented. Then it presents the problem statement, which is the fairness issue that exists among multi-threaded applications. In Section 3, some related works are introduced, including their deficiencies. Then, the improving scheme is discussed. We present a solution based on the discussion. In Section 4, the experiment is carried out to illustrate the problem statement and evaluate the new scheduler algorithm. Finally, a conclusion is made in Section 5.

II. COMPLETELY FAIR SCHEDULER

A. Basic Algorithm of CFS

"80% of CFS design can be summed up in a single sentence: CFS basically models an 'ideal, precise multi-tasking CPU' on real hardware." [7] The main idea of CFS is to keep the balance (fairness) of CPU time that shared by tasks. It means two processes, which have the same property, priority and calculated amount, will stop running almost at the same time if they execute concurrently. So, the unbalanced task should obtain CPU resource, and execute.

In order to keep balance, virtual running time (vruntime)

is introduced by CFS. Each process has on vruntime, and it increases monotonically. The increasing speed of process vruntime is decided by its priority. The lower process priority is, the faster vruntime increases. And all processes chased each other. CPU will always run the process which has the minimal vruntime. It is compensation for dropped one, and this is fairness.

CFS proves fairness via weight of each scheduling entity (process or thread), which is decided by priority. Linux kernel gets the weight of each scheduling entity through a transformation as nice-prio-weight.

- **nice-prio**

```
#define NICE_TO_PRIO(nice) (MAX_RT_PRIO
+ (nice) + 20)
```

Where MAX_RT_PRIO=100, the value domain of nice is [-20,19], so we can conclude the value domain of prio is [100,139].

- **prio-weight**

```
static const in prio_to_weight[40] = {
/*-20*/88761, 71755, 56483, 46273, 36291,
...
/*15*/ 36, 29, 23, 18, 15,
};
```

This is an experiential formula, we do not show them all here for space limited, and the full matrix can be found in the source code of Linux 2.6.33 kernel. So we can tune weight via nice.

The vruntime calculated via following formula after the value of weight is gotten.

$$vruntime += \Delta_{exec} \times \frac{NICE_0_LOAD}{load.weight} \quad (1)$$

Where Δ_{exec} is the running time of current process, NICE_0_LOAD is a constant factor, which refers to the default weight of process in Linux.

All runnable tasks are sorted in a red-black tree by value of a “se->vruntime - cfs->min_vruntime” key which is calculated in function `entity_key`. A red-black tree is a self-balancing binary tree where the leftmost leaf has the smallest value because at each node, the smaller value is inserted into the left child, and the larger value is inserted into the right child[8].

B. The Shortcoming of CFS

Although the idea of CFS is quite advanced, and can make the scheduling fair in the most scenarios, there are still some exceptions. For example, user application can create a lot of child process via `fork()`, so that it can obtain more CPU resource and execute faster. In order to solve this problem, CFS introduces group scheduling.

However, the granularity of CFS scheduling is thread, so the same problem can occurred in multi-threaded applications. For example, there are three processes: A, B, and C, created 1, 2 and 8 threads respectively. The CPU resource allocation

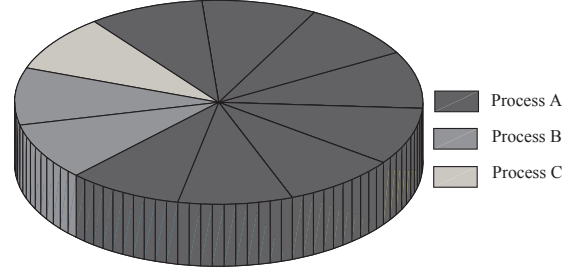


Fig. 1: CFS CPU resource allocation chart

is illustrated in Fig. 1.

In Fig. 1, the process C receives an unfair amount of time slice due to its excessive number of threads. Here, the process which contains more than the optimal number of threads is termed as “greedy” threaded process. Although each thread receives the same amount of CPU time slice (all threads have the same nice value), Process C dominates most of the bandwidth because the ratio of the number of threads in process C to process B and A is 8:2:1. This issue needs to be addressed because this could be an advantage to software developers to spawn more additional unwanted threads than required so that their program can obtain a larger share of CPU resources when there are other programs with lower number of threads executing concurrently.

III. IMPROVING SCHEME

A. Discussion of Related Work

Actually, there are two types of schedule algorithms in current Linux kernel, one is CFS, and another is real-time schedule algorithm. Macro SCHED_NORMAL refers to CFS, while SCHED_FIFO and SCHED_RR refer to the real-time scheduler. One of the most important real-time scheduler is EDF (Earliest Deadline First)[9]. There are a lot of existing state-of-the-art schedulers, such as global-EDF pfair[10] and LitmusRT[11]. All of the schedulers mentioned above aim to “real-time” capability, which means the real-time process (or high priority process) should response in time. However, what we discussed here is about the fairness of Linux scheduler, not the response time.

Related work is discussed in [12], where the fairness among thread groups was suggested. Similarly, the process level fair-sharing algorithm has also been implemented in existing systems such as Aurema ARMTEchs Workload Management Solutions for Citrix Metaframe Optimization[13] and Solaris 10[14]. Reference [15] introduced a solution called PFS (Process Fair Scheduler), it takes advantage of group scheduling, and upgrades the scheduling granularity from thread to process. The weight calculation formula in PFS is:

$$se \rightarrow load.weight' = \frac{se \rightarrow load.weight}{\alpha} \quad (2)$$

Where α is the thread number of the current process. The CPU resource allocation with PFS algorithm is illustrated in Fig. 2.

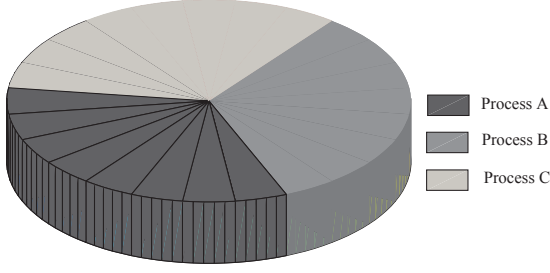


Fig. 2: PFS CPU resource allocation chart

We can see, all processes are treated completely fair, they share the CPU resource equally. However, there are two problems in this method:

- 1) Due to the growing availability of chip multiprocessors (CMP), software applications are encouraged to be designed using multiple threads so that the benefit of thread-level parallelism (TLP) can be exploited. While PFS does not “encourage” the well threaded programs, on contrary, the performance of these programs may descend because of the context switch between threads.
- 2) The “greedy” threaded process can create a lot of threads, resulting in frequent context switches, consuming significant CPU resource; in consequence the whole system performance descends sharply. PFS can do nothing about this situation.

B. Basic Scheme of Improving CFS

Therefore, the improving CFS algorithm should fulfill the following three conditions:

- 1) Encourage the well threaded program, (the definition of “well threaded” program can be found in reference [12]), increase their weight.
- 2) Punish the greedy threaded program, decrease their weight.
- 3) The algorithm of scheduler should be elegant and efficient, there should not be complex calculations, and otherwise it is no need to modify the scheduler.

According to the analysis of reference [15], the optimal number of threads in multi-threaded program equals to the number of available processing cores (N_0) in the system. Therefore, the improving CFS algorithm, we call it DW-CFS (Dynamic Weight Completely Fair Scheduler), its kernel idea is dynamically tuning the weight of process, based on the number of threads that the process created. N_0 becomes the boundary of encouragement and punishment.

We change the formula of weight calculation to:

$$se \rightarrow load.weight' = \alpha \times se \rightarrow load.weight \quad (3)$$

Where $\alpha = f(N_t, N_0, \beta)$, N_t is the number of threads that the current scheduling entity contains, N_0 is the number of available processing cores, β is the factor of weight tuning, its a constant, and represents to the extra weight gain that one multi-threaded program can get.

Then (1) becomes:

$$vruntime+ = delta_exec \times \frac{NICE_0_LOAD}{\alpha \times load.weight} \quad (4)$$

C. Analysis of Function α

Firstly, we can describe this question in mathematics:

$f(N_t, N_0, \beta)$, where $N_t, N_0 \in N^+, \beta > 1$

Conditions:

- 1) $\forall n, m \in N^+, n < m \leq N_0 \Rightarrow f(n, N_0, \beta) < f(m, N_0, \beta)$
- 2) $\forall n, m \in N^+, n > m > N_0 \Rightarrow f(n, N_0, \beta) < f(m, N_0, \beta)$
- 3) $f(1, N_0, \beta) = 1$
- 4) $f(N_0, N_0, \beta) = \beta$
- 5) $\lim_{N_t \rightarrow \infty} f(N_t, N_0, \beta) = 0$

Considering separate β from function α , for the given N_0 , we can transform $f(N_t, N_0, \beta)$ to $F(N_t) + \beta$, according to condition 4, $F(N_0) = 0$.

Because $f(N_t, N_0, \beta) \in (0, \beta]$, so $F(N_t) \in (-\beta, 0]$,

Therefore, we can assume that

$$F(N_t) = -\left| \frac{N_t - N_0}{\gamma} \right| \quad (5)$$

Where γ is the function of N_t .

According to condition 5:

$$\lim_{N_t \rightarrow \infty} F(N_t) = -\beta \quad (6)$$

And

$$F(N_t) = -\left| \frac{N_t - N_0}{\gamma} \right| = -\left| \frac{1 - \frac{N_0}{N_t}}{\frac{\gamma}{N_t}} \right| \quad (7)$$

Because $\lim_{N_t \rightarrow \infty} (1 - \frac{N_0}{N_t}) = 1$

So

$$\lim_{N_t \rightarrow \infty} \frac{\gamma}{N_t} = \frac{1}{\beta} \quad (8)$$

Then, we can assume that

$$\gamma = \frac{N_t}{\beta} + \varphi \quad (9)$$

Where φ is the function that is irrelevant to N_t .

Then

$$F(N_t) = -\left| \frac{N_t - N_0}{\frac{N_t}{\beta} + \varphi} \right| \quad (10)$$

According to condition 3, $F(1) = 1 - \beta$, we can get

$$F(1) = -\left| \frac{1 - N_0}{\frac{1}{\beta} + \varphi} \right| = 1 - \beta \quad (11)$$

Because $N_0 \in N^+, \beta > 1$

So

$$\left| \frac{1}{\beta} + \varphi \right| = \frac{N_0 - 1}{\beta - 1} \quad (12)$$

Equal to

$$\varphi = -\frac{1}{\beta} \pm \frac{N_0 - 1}{\beta - 1} \quad (13)$$

Take it to (10), we can get

$$F(N_t) = - \left| \frac{(\beta - 1) * \beta * (N_t - N_0)}{(\beta - 1) * (N_t - 1) \pm \beta * (N_0 - 1)} \right| \quad (14)$$

Finally, according to condition 1 and condition 2:

$$F(N_t) = - \left| \frac{(\beta - 1) * \beta * (N_t - N_0)}{(\beta - 1) * (N_t - 1) + \beta * (N_0 - 1)} \right| \quad (15)$$

Therefor

$$f(N_t, N_0, \beta) = - \left| \frac{(\beta - 1) * \beta * (N_t - N_0)}{(\beta - 1) * (N_t - 1) + \beta * (N_0 - 1)} \right| + \beta \quad (16)$$

Assume $N_0 = 4, \beta = 1.1$, then the curve plot of function α is illustrated in Fig. 3.

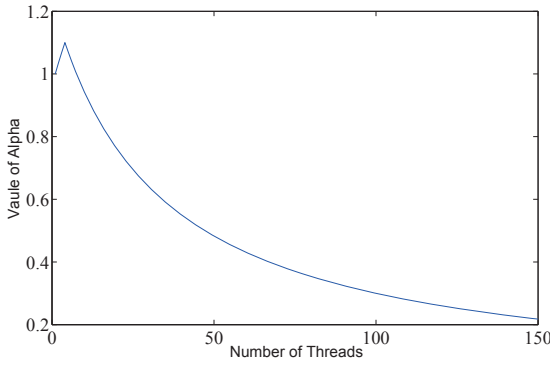


Fig. 3: Weight tuning function ($N_0 = 4, \beta = 1.1$)

Furthermore, there is a problem that, when $N_0 = 1$, $f(N_t, N_0, \beta) = 0$ is an identical equation. That makes no sense of scheduler. So if $N_0 = 1$, which means there is only one available processing core in the system, the DW-CFS should be forbidden.

IV. EVALUATION AND ANALYSIS

In order to demonstrate the problem of unfairness, and evaluate the effectiveness of DW-CFS algorithm, we need to do performance experience. There is some specific simulation software of scheduling performance, such as LinSched[16]. However, the performance in the real work scenario is much more persuasive. So we are utilizing a multi-threaded program executing in a multi-processor environment.

We installed two kernels: one for testing CFS and one for testing DW-CFS. We download the 2.6.33 source code from kernel.org, and made two copies, one is left unmodified, while another is patched with DW-CFS.

A. Evaluation Environment

TABLE I: EXECUTION ENVIRONMENT

Processor	Intel® Core™ i7-2600 @ 3.40GHz
Main Memory	2*4 GB DDR3 @1600 9-9-9-24
OS/Kernel	Red Hat Enterprise Linux 5 x64/2.6.33

Our CPU has 4 physical cores on chip and provides 8 threads capability via hyper-threading technology. However the hyper-threading is not a real parallel operation and we are running a CPU-bound program to test, which make no sense of hyper-threading. So we turn off the hyper-threading and the number of available processing cores equal to 4.

B. Evaluation Method

The program chosen was a pi (π) calculation program which utilizes POSIX thread (pthreads) libraries that were available as part of the GNU C compiler distribution. There are two typical algorithms of calculation, one is Monte Carlo method, and another is discrete integration method. The Monte Carlo method calculates via statistics cannot get a precise answer, and the consumption is different from each other. So we use the second one, and its principle is showed below:

$$\int_0^1 \frac{1}{1+x^2} dx = \arctan(x) \Big|_0^1 = \pi/4 \quad (17)$$

Set

$$f(x) = \frac{4}{1+x^2} \quad (18)$$

Then

$$\int_0^1 f(x) dx = \pi \quad (19)$$

$$\pi = \sum_{i=1}^N f\left(\frac{2 \times i - 1}{2 \times N}\right) \times \frac{1}{N} = \frac{1}{N} \times \sum_{i=1}^N f\left(\frac{i - 0.5}{N}\right) \quad (20)$$

The program to calculate π is a simple parallel computation intensive program that does not require communication between the threads and has only one shared variable to update. In (20), we can conclude that the larger N is, the more precise answer can be, and the longer execution consumption will be. The N value is made very large to ensure that the approximation program takes at least a few seconds to complete. This granularity is required as we use the time command to conduct the timing. In our program, we set the N value to 10^9 .

The following test was conducted under normal desktop conditions with no other computation intensive applications running. The two programs were initiated through a shell script where the programs were executed with their priority settings fixed to the same value (nice 0). The parameter β of DW-CFS is set to 1.1. The evaluations were done for 2 programs executing concurrently with one of the programs fixed at single thread (Program A) and the other had its thread count varied N_t threads, where $N_t \in \{1, 2, 3, 4, 5, 6, 7, 8, 12, 16, 32\}$ (Program B). Each simulation was done 100 times and the average timing was used.

C. Results Analysis

Fig 4(a) shows that as the increasing N_t , the threads number of process B, the execution time of process B is decreasing. As we use 4 cores CPU, and turns off hyper-threading, so when $N_t = 4$, time decreasing becomes minor, but still decreasing. This shows that multi-threaded process

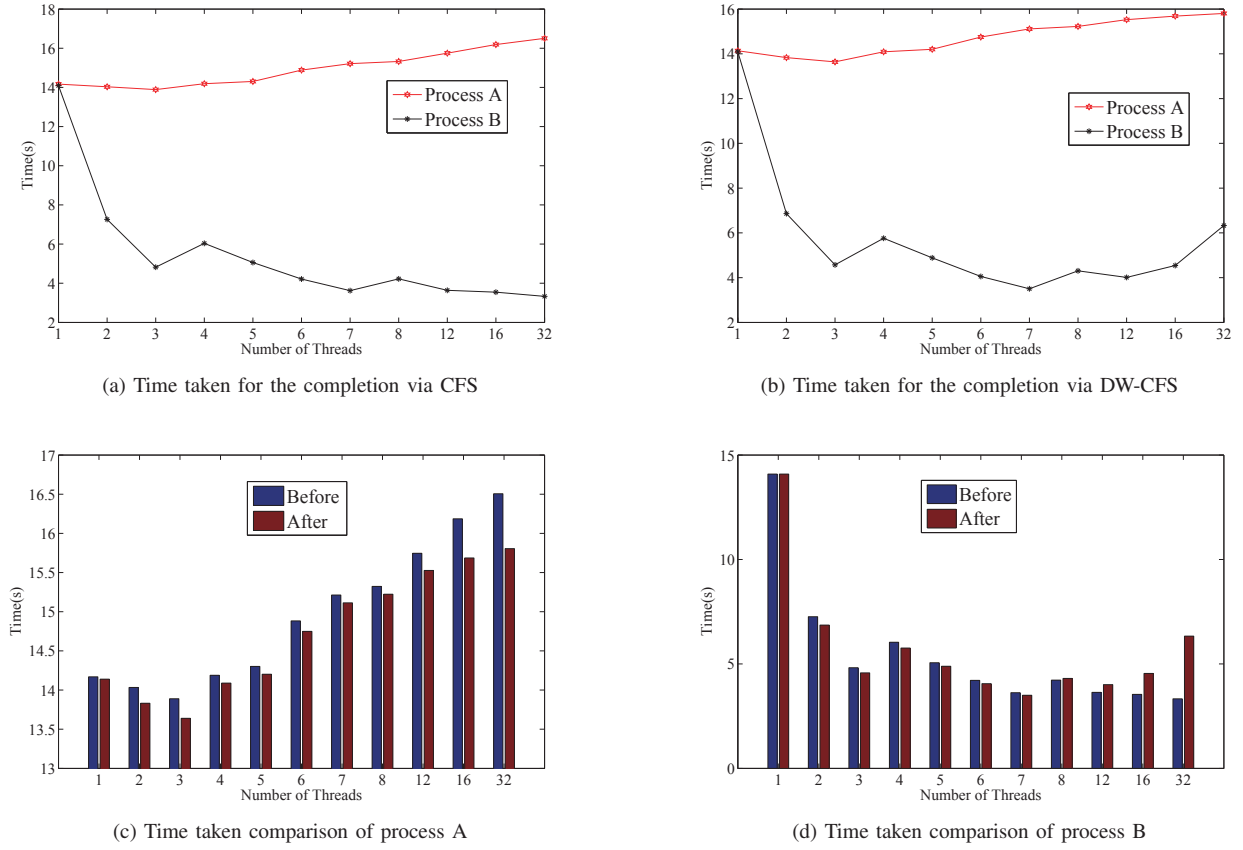


Fig. 4: Results of the evaluation

can dominate the CPU resource, and make unfair scheduling. Meanwhile, the execution time of process A is increasing with the increasing of N_t , once again proved the fairness issue does exist.

There are two points should be paid attention to. First, when $N_t < 4$, the execution time of process A is decreasing as well, this is because of the Turbo Boost technology of CPU, it can improve the performance of single task. Second, the execution time curve of process B has two inflexions at $N_t = 4$ and $N_t = 8$. This is because our CPU has 4 cores, when N_t equal to 4, 8, 12 and so on, process B will allocate all CPU cores, but there is process A executing concurrently, and it will increase the scheduling time, so the execution time of process B increased. This happens obviously when $N_t = 4$ and $N_t = 8$, as N_t increasing, the scheduling becomes frequently, this impact will be not obvious.

Fig. 4(b) shows that, DW-CFS can prevent process B from dominating CPU resource when N_t becomes too large. "Punishment" works well, and process A also takes advantage of it, its execution time curve becomes more smooth.

In Fig. 4(c) and Fig. 4(d), when $N_t \in [1, 3]$, process B make good use of multi-threads, so DW-CFS increase its weight, it can be scheduled faster, and the execution time is smaller than that with CFS. The execution time of process A

is decreased because process B ends. When $N_t \in [4, 7]$, the advantage of threads increasing is no longer obvious, but it is still better than CFS. When $N_t = 8$, $\alpha = 0.99$, "punishment" begins, the execution time of process B is increasing. Last, when $N_t \in [8, 32]$, the degree of "punishment" becomes bigger and bigger, and the effect is remarkable. It proves that the algorithm is working. Meanwhile, process A gets more CPU resource; its execution time is smaller than before.

However, because of the threads context switch consumption and Turbo Boost, the effect of the DW-CFS is counteracted in a certain extent.

V. CONCLUSIONS

In this paper, according to analyze the basic principle of CFS, the fairness issue in a multi-threads environment was presented, and related works were discussed, then the scheme of improving method was introduced. According to the discussion before, we designed DW-CFS algorithm, and analyzed the weight tuning function, gave one available function. This method dynamically tuned the weight of process, based on the number of threads that the process created. So the well threaded program can get priority of scheduling, and greedy threaded program will be punished. The experiment showed this algorithm can reduces the impact of greedy threaded

program sharply. The limitation of the proposed solution is that the effect of encourage is minor.

REFERENCES

- [1] A. Silberschatz, P.B. Gailvin, G. Gagne, Operating System Concepts, 9th ed., John Wiley Sons, 2012.
- [2] M. Wolfgang, Professional Linux Kernel Architecture, Wiley Publishing, Inc., 2008.
- [3] R. Love, Linux Kernel Development, 2nd ed, Noval Press, 2010.
- [4] M.T. Jones. Inside the Linux 2.6 Completely Fair Scheduler. <http://www.ibm.com/developerworks/linux/library/l-completely-fair-scheduler/index.html>
- [5] I. Molnar, Modular Scheduler Core and Completely Fair Scheduler. <http://lwn.net/Articles/230501/>
- [6] I. Molnar, CFS Updates, http://kerneltrap.org/Linux/CFS_Updates/
- [7] J. Andrew, Interview: Ingo Molnar. <http://kerneltrap.org/?q=node/517>
- [8] H.C. Thomas, et al, Introduction to Algorithms, The MIT Press, 2009.
- [9] D. Faggioli, et al, "An EDF scheduling class for the Linux kernel", Proceedings of the Real-Time Linux Workshop, 2009.
- [10] J. Anderson, A. Srinivasan, "Early-release fair scheduling", Proceedings of the 12th Euromicro Conference on Real-Time Systems, 35-42, 2000.
- [11] L. Abeni, G. Lipari, Implementing Resource Reservations in Linux, Real Time Linux Workshop, 2002.
- [12] <http://kerneltrap.org/mailarchive/linuxkernel/2007/4/18/78624>
- [13] Aurema ARMTech, Workload Management Solutions for Citrix MetaFrame Optimization, http://www.responseunit.com/amherst/armtech/armtech_citrix_white_paper.pdf
- [14] Solaris 10+ Resource Management, <http://www.princeton.edu/~unix/Solaris/troubleshoot/resmgmt.html>
- [15] S.W. Chee, T. Ian, D.K. Rosalind, et al, "Towards Achieving Fairness in the Linux Scheduler", Operating Systems Review (ACM), 42(5): 34-43, 2008.
- [16] M.C. John, P.B. Dan, L. Tong, et al, "LinSched: The Linux Scheduler Simulator", ISCA PDCCS, 171-176, 2008.